

08Bikes-Model-Selection

October 5, 2021

1 Data Science: Bridging Principle and Practice

1.1 Part 8: Model Selection (Bike Sharing case study)

```
<div style="float:left;width:40%">
    
</div>
<div style="float:left;width:40%">
    
</div>
```

1.1.1 Table of Contents

Section ??

```
<li><a href="#section8">Model Selection</a>
    <ol>
        <br>
        <li> <a href="#section8a">Using a Validation Set</a></li>
        <br>
        <li><a href="#section8b">Improving the Model</a></li>
    </ol>
</li>
</ol>
```

```
In [1]: # run this cell to import some necessary packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
%matplotlib inline
import seaborn as sns
from gofer.ok import check
import warnings
warnings.filterwarnings('ignore')
import ipywidgets as widgets
from scipy.linalg import lstsq
import ipywidgets as widgets
```

```

from sklearn.metrics import mean_squared_error as mse
from sklearn.model_selection import train_test_split
from IPython.display import display, clear_output
from scripts.bike_model_widget import *

```

1.2 Case Study: Capital Bike Share (review)

Bike-sharing systems have become increasingly popular worldwide as environmentally-friendly solutions to traffic congestion, inadequate public transit, and the "last-mile" problem. Capital Bikeshare runs one such system in the Washington, D.C. metropolitan area.

The Capital Bikeshare system comprises docks of bikes, strategically placed across the area, that can be unlocked by *registered* users who have signed up for a monthly or yearly plan or by *casual* users who pay by the hour or day. They collect data on the number of casual and registered users per hour and per day.

Let's say that Capital Bikeshare is interested in a **prediction** problem: predicting how many riders they can expect to have on a given day. [UC Irvine's Machine Learning Repository](#) has combined the bike sharing data with information about weather conditions and holidays to try to answer this question.

```

In [2]: # run this cell to load the data
        bikes = pd.read_csv("data/day_renamed_dso.csv", index_col=0)

        # show the first five rows
        bikes.head()

```

```

Out[2]:

```

	date	season	year	month	is holiday	week day	is work day	\
instant								
1	2011-01-01	spring	2011	1	no	saturday	no	
2	2011-01-02	spring	2011	1	no	sunday	no	
3	2011-01-03	spring	2011	1	no	monday	yes	
4	2011-01-04	spring	2011	1	no	tuesday	yes	
5	2011-01-05	spring	2011	1	no	wednesday	yes	

	weather	temp	felt temp	humidity	windspeed	casual riders	\
instant							
1	2	0.344167	0.363625	0.805833	0.160446	331	
2	2	0.363478	0.353739	0.696087	0.248539	131	
3	1	0.196364	0.189405	0.437273	0.248309	120	
4	1	0.200000	0.212122	0.590435	0.160296	108	
5	1	0.226957	0.229270	0.436957	0.186900	82	

	registered riders	total riders
instant		
1	654	985
2	670	801
3	1229	1349
4	1454	1562
5	1518	1600

A few of the less straight-forward columns can be described as follows: - **instant**: record index - **is 2012** : 1 if the date is in 2012, 0 if the date is in 2011 - **is holiday** : 1 if day is a holiday, 0 otherwise - **is work day** : 1 if day is not a weekend or holiday, otherwise 0 - **weather** : - 1: Clear, Few clouds, Partly cloudy, Partly cloudy - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds - 4: Heavy Rain + Ice Pellets + Thunderstorm + Mist, Snow + Fog - **temp** : Normalized temperature in Celsius. The values are derived via $(t-t_{\min})/(t_{\max}-t_{\min})$, $t_{\min}=-8$, $t_{\max}=+39$ (only in hourly scale) - **felt temp**: Normalized feeling temperature in Celsius. The values are derived via $(t-t_{\min})/(t_{\max}-t_{\min})$, $t_{\min}=-16$, $t_{\max}=+50$ (only in hourly scale) - **humidity**: Normalized humidity. The values are divided to 100 (max) - **windspeed**: Normalized wind speed. The values are divided to 67 (max) - **casual**: count of casual users - **registered**: count of registered users - **total riders**: count of total rental bikes (casual + registered)

1.3 8. Model Selection

1.3.1 8a. Using a Validation Set

In Notebook 06 (Exploratory Data Analysis), we talk about the need to split our data into a *training set* and a *test set*. A test-train split reduces the chance of **overfitting**- that is, the chance that our model learns from the data so well that it makes incredibly accurate predictions for items in the training data, but makes wildly inaccurate predictions for data it hasn't been trained on. Another explanation of the test-train split and overfitting can be found at [this link](#).

In this notebook, we want to compare different linear models to one another to find a "best" model. One major indicator of a "best" model is how well it makes predictions for data it hasn't been trained on.

A test set would be perfect for seeing how different models performs on new data to select the best model. But, when we compare multiple models to one another and make predictions for the test data multiple times, we get a new problem- we might overfit to the *test set*! In a perfect world, the test set is only ever used once, after you've selected your model, to get one final measure of how well it might generalize to new data.

We can get around this by using a **validation set**. A validation set is used during model selection. It acts a bit like a test set- the model isn't trained on data in the validation set, so having the model make predictions for the validation data can help us see how well our model performs on new data. There are several different methods of creating a validation set, but we'll use a simple split where we randomly select 20% of the rows in the training set.

Run the next cell to split the data into training, validation, and test sets. After this cell is run, 20% of the original data will be in the test set, 20% of the original data will be in the validation set, and the remaining 60% will be in the training set.

```
In [3]: # set the random seed
        np.random.seed(28)

        # set aside 20% of the data for testing
        bikes, bike_test = train_test_split(bikes, train_size=0.8, test_size=0.2)

        # set aside 25% of the remaining data for validation
        bike_train, bike_val = train_test_split(bikes, train_size=0.75, test_size=0.25)
```

1.4 8b. Finding The Best Model

In notebook 07, we created a linear regression model where we tried to predict the total number of riders on a given day based on the temperature, season, and whether or not the day was a work day. Ultimately, our model was not very accurate. Our challenge now is to build a better model.

Running the next cell will generate a widget that will train a model according to your chosen features and return several metrics for evaluating the model.

Note: The code that trains the model and makes the graphs is somewhat lengthy and not important for the purposes of this assignment, so it has been put in a separate file. If you're interested, you can explore "bike_model_widget.py" in the "scripts" folder. The code that creates the models and graphs is very similar to the code you saw and wrote in Notebook 07.

The widget contains the following: - the grey rectangular buttons represent features that can be included in your linear model, as well as an intercept term. Clicking on one or more buttons will include those features in the model. - the circular buttons let you choose which response variable you want your model to predict: total riders, registered riders, or casual riders. You may only choose one response variable.

Based on your selections, the widget will generate: - the RMSE (root mean squared error) for the model's predictions for the training data and the validation data - β : the coefficients for your linear regression model - scatter plots with best-fit lines for the predicted values plotted against the actual values, for both the training data (in blue) and the validation data (in gold) - scatter plots with best-fit lines for the errors (residuals) plotted against the actual values of the response variable, for both the training data (in blue) and the validation data (in gold). These scatter plots also have black horizontal lines at zero to help you see whether the model tends to over- or under-estimate.

In [4]: *# run this cell to generate the widget*

```
display(expl_buttons, intercept, response_radio,out)
```

```
Box(children=(ToggleButton(value=False, description='season'), ToggleButton(value=False, descr
```

```
ToggleButton(value=False, description='intercept')
```

```
RadioButtons(description='Response variable:', options=('total riders', 'casual riders', 'regi
```

```
Output(layout=Layout(height='800px'), outputs=({'output_type': 'stream', 'text': 'Please select
```

EXERCISE:

Use the widget to look for more accurate models. Try at least 10 combinations of features, and make sure to track which model seems to be performing the best.

<p>Note: it may take a few seconds for the graphs to update after you click the buttons.</p>

<p>Record the parameters for your best model in the cell below. </p>

<code>best_expl_vars</code> should be a list of the names of the explanatory variables

<code>best_intercept</code> should be assigned to <code>True</code> if your best model

<code>best_response_var</code> should be assigned to the name of the response variable
<code>best_beta</code> should be assigned to the beta vector; the list of coefficients


```
In [5]: # list the explanatory variables used in your best model in a list
# variable names should be in quotation marks, and each name should be
# separated by commas inside the square brackets
best_expl_vars = ["year", "is work day", "temp", "humidity"]

# replace the ellipses with True if your model used an intercept term
# otherwise, replace the ellipses with False
best_intercept = True

# replace the ellipses with the name of the response variable
best_response_var = "casual riders"

# replace the ellipses with a list of the coefficients in the beta vector.
# you can copy and paste the beta values that the widget prints out, but
# you will need to add commas in between the different values
best_beta = [-6.25348202e+05, 3.11226601e+02, -8.27344883e+02, 2.13173555e+03,
             -5.09492719e+02]

In [6]: # run this cell to check your answers for common errors
check("tests/best-model.ok")
```

Out[6]: <gofer.ok.OKTestsResult at 0x7f0a600254e0>

QUESTION: What was your approach to finding a better model? Explain which variables you tried and why, as well as what metrics showed it was the "better". Reference the scatter plots, fit lines, RMSE, etc, and record the explanatory and response variables for your best model.

ANSWER:

Finally, run the next cell to try your best model on the test data.

```
In [7]: # run this cell to try your best model on the test data.
# if you get an error, make sure you ran the cell where you saved
# the parameters of your best model
X_test = make_X(bike_test, best_expl_vars)
y_test = bike_test[best_response_var]

if not best_intercept:
    X_test.drop("intercept", axis=1, inplace=True)

# make predictions
pred_test = X_test @ best_beta

# generate plots
f, [ax1, ax2] = plt.subplots(2, figsize=(12, 12))

sns.regplot(x=y_test, y=pred_test, ax=ax1, color="#ED4E33")
```

```

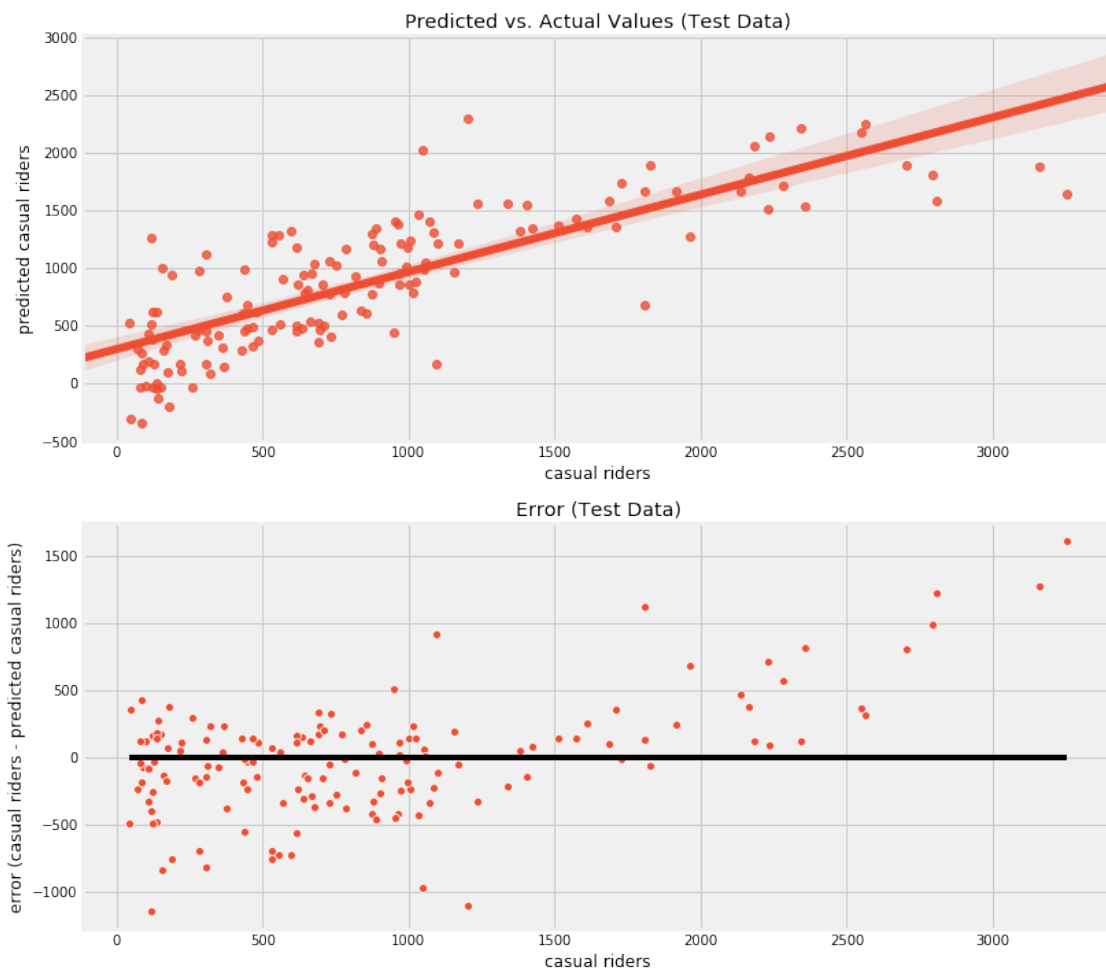
ax1.set_xlabel(best_response_var)
ax1.set_ylabel("predicted {}".format(best_response_var))
ax1.set_title("Predicted vs. Actual Values (Test Data)")

sns.scatterplot(x=y_test, y=y_test - pred_test, ax=ax2, color="#ED4E33")
ax2.set_xlabel(best_response_var)
ax2.set_ylabel("error ({} - predicted {})".format(best_response_var, best_response_var))
ax2.set_title("Error (Test Data)")
ax2.hlines(y=0, xmin=min(y_test), xmax=max(y_test))

# calculate rmse
print("Test data RMSE = {}".format(np.sqrt(mse(pred_test, y_test))))

```

Test data RMSE = 428.51220287918585



QUESTION: How did your model perform on the test data compared to how it performed on

the training and validation data? Given what you know about how models "learn" from data, are the results you saw in line with your expectations?

ANSWER:

References

- Bike-Sharing data set from University of California Irvine's Machine Learning Repository <https://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset>
- Portions of text and code adapted from Professor Jonathan Marshall's Legal Studies 190 (Data, Prediction, and Law) course materials: [lab 2-22-18, Linear Regression](#) (Author Keeley Takimoto) and [lab 3-22-18, Exploratory Data Analysis](#) (Author Keeley Takimoto)
- "Capital Bikeshare, Washington, DC" header image by [Leeann Caferatta](#) licensed under [CC BY-ND 2.0](#)